we must modify the Bike class and specify the new class. This makes Bike class very brittle and difficult to manage.

One way to avoid the changes to Bike class is to take the dependencies inside its constructor parameters.

For example,

```
export class Bike {
    public engine: BikeEngine;
    public tires: BikeTires;
    public description = 'No DI';

    constructor(BikeEngine engine, BikeTires tires) {
        this.engine = engine;
        this.tires = tires;
    }

    public ride() {
 return `${this.description} bike with ${this.engine.cylinders} cylinders and ${this.tires.make} tires.`;}}
```

Now on we can easily change the dependency of Bike class. Every time we need the instance of Bike we can create it as follows

```
      Let bike = new Bike(new BikeEngine(), new BikeTire());
```

If we need custom Bike instance with special engine we could do so as,

```
Let bike = new Bike(new SpecialBikeEngine(), new BikeTire());
```

The bike class is untouched. It can also share the dependency objects with others if they are heavy and takes considerable resources.

The most important benefit of having externally injected dependencies is that, testing of the given class becomes much easier. We can provide MockEngine and MockTires which do nothing except let Bike satisfy its dependencies. It enable us to write simple and independent test cases.

```
class MockEngine extends BikeEngine { cylinders = 4; }
class MockTires  extends BikeTires  { make = 'MRF'; }
```

// Test car with 4 cylinders and MRF tires.

let bike= new Bike(new MockEngine(), new MockTires());